

Guide Pratique

Git & GitHub pour débutants

Créer un compte GitHub, créer un dépôt (repository),
le lier à votre ordinateur, et faire votre premier commit

Ce guide est destiné aux débutants complets. Chaque étape est expliquée simplement, avec les commandes exactes à taper dans **Git Bash**. Suivez les étapes dans l'ordre, une par une.

Sommaire

- Étape 1 — Créer un compte GitHub
- Étape 2 — Installer Git (Git Bash)
- Étape 3 — Configurer Git avec votre identité GitHub
- Étape 4 — Créer un repository (dépôt) sur GitHub
- Étape 5 — Créer votre dossier de travail local
- Étape 6 — Lier votre dossier local au repository GitHub (git init + remote)
- Étape 7 — Créer un fichier hello.js
- Étape 8 — git add : préparer les fichiers
- Étape 9 — git commit : enregistrer votre travail
- Étape 10 — git push : envoyer votre travail sur GitHub
- Étape 11 — git pull : récupérer les changements
- Résumé — Les commandes à retenir

1

Créer un compte GitHub

GitHub est un site web qui permet d'héberger vos projets de code, de garder un historique de toutes vos modifications, et de collaborer avec d'autres développeurs.

Pour créer votre compte :

- Allez sur le site : **<https://github.com>**
- Cliquez sur **Sign up** (S'inscrire)
- Choisissez un nom d'utilisateur (username) simple et professionnel (par exemple : votrenom-dev)
- Entrez votre adresse email et un mot de passe
- Vérifiez votre email en cliquant sur le lien envoyé par GitHub

Note : gardez bien en tête votre **nom d'utilisateur** et votre **email**, vous en aurez besoin à l'étape 3.

2

Installer Git (Git Bash)

Git est un logiciel installé sur votre ordinateur qui permet de suivre l'historique de votre code. GitHub, lui, est le site web où vous envoyez ce code. Les deux sont différents mais fonctionnent ensemble.

- Allez sur : **<https://git-scm.com/downloads>**
- Téléchargez la version pour Windows (ou votre système)
- Installez le programme en laissant les options par défaut (cliquez sur Next à chaque étape)
- Une fois installé, cherchez **Git Bash** dans votre menu Démarrer et ouvrez-le. C'est un terminal noir : c'est ici que vous allez taper toutes les commandes de ce guide.

Pour vérifier que l'installation a fonctionné, tapez :

```
git --version
```

Vous devez voir s'afficher un numéro de version, par exemple : git version 2.45.0

3

Configurer Git avec votre identité GitHub

Avant d'utiliser Git, vous devez lui dire qui vous êtes. Ces informations seront attachées à chaque enregistrement (commit) que vous ferez. Utilisez le **même nom et le même email que sur votre compte GitHub**.

Dans Git Bash, tapez ces deux commandes (une par une) :

```
git config --global user.name "VotreNomUtilisateur"  
git config --global user.email "votre.email@exemple.com"
```

Pour vérifier que c'est bien enregistré :

```
git config --global user.name  
git config --global user.email
```

Astuce : cette configuration ne se fait **qu'une seule fois** sur votre ordinateur. Vous n'aurez plus besoin de la refaire pour vos prochains projets.

4

Créer un repository (dépôt) sur GitHub

Un **repository** (ou "repo") est un dossier hébergé sur GitHub qui va contenir votre projet.

- Connectez-vous sur **<https://github.com>**
- Cliquez sur le bouton vert **New** (ou le + en haut à droite, puis **New repository**)
- Donnez un nom à votre repository, par exemple : **premier-projet-git**
- Laissez-le en **Public**
- Ne cochez PAS "Add a README file" pour l'instant (pour garder ce guide simple)
- Cliquez sur **Create repository**

GitHub va afficher une page avec un lien qui ressemble à ceci :
<https://github.com/votrenom/premier-projet-git.git>
Gardez cette page ouverte, vous en aurez besoin juste après.

5

Créer votre dossier de travail local

Maintenant, créons un dossier sur votre ordinateur qui va contenir votre projet. Dans Git Bash, naviguez vers l'endroit où vous voulez travailler (par exemple le Bureau) puis créez le dossier :

```
cd Desktop
mkdir premier-projet-git
cd premier-projet-git
```

Explication : `cd` permet de se déplacer dans un dossier, `mkdir` crée un nouveau dossier.

6

Lier votre dossier local au repository GitHub

Toujours dans le même dossier, transformez-le en dépôt Git :

```
git init
```

Cette commande crée un dossier caché `.git` qui va suivre l'historique de votre projet.

Ensuite, liez ce dossier local au repository que vous avez créé sur GitHub (remplacez le lien par le vôtre, copié à l'étape 4) :

```
git remote add origin https://github.com/votrenom/premier-projet-git.git
```

Pour vérifier que le lien est bien fait :

```
git remote -v
```

Vous devez voir apparaître le lien de votre repository GitHub, avec (fetch) et (push).

7

Créer un fichier hello.js

Créons maintenant un petit fichier JavaScript. Vous pouvez ouvrir votre dossier avec VS Code, ou utiliser directement Git Bash :

```
echo 'console.log("Hello GitHub");' > hello.js
```

Cette commande crée un fichier `hello.js` contenant le code `console.log("Hello GitHub");`. Vous pouvez aussi créer ce fichier manuellement avec VS Code si vous préférez, le résultat est le même.

Vérifiez le contenu du fichier avec :

```
cat hello.js
```

8 git add : préparer les fichiers

Avant d'enregistrer un changement, il faut dire à Git QUELS fichiers on veut inclure. C'est ce qu'on appelle "mettre en zone de préparation" (staging).

```
git add hello.js
```

Astuce : pour ajouter TOUS les fichiers modifiés d'un coup, vous pouvez utiliser :

```
git add .
```

Pour vérifier l'état de votre dépôt à tout moment :

```
git status
```

Le fichier doit apparaître en vert, sous la section "Changes to be committed".

9 git commit : enregistrer votre travail

Un **commit** est comme une photo ("instantané") de votre projet à un moment donné, accompagnée d'un message qui explique ce que vous avez fait.

```
git commit -m "my first commit"
```

Explication : `-m` permet d'ajouter directement un message. Choisissez toujours un message clair qui résume ce que vous avez changé.

10 git push : envoyer votre travail sur GitHub

Maintenant que votre commit est enregistré localement, envoyons-le sur GitHub pour qu'il soit visible en ligne.

```
git push -u origin main
```

La première fois, Git Bash peut vous demander de vous connecter à votre compte GitHub (une fenêtre peut s'ouvrir dans votre navigateur). Autorisez la connexion.

Note : `-u origin main` ne sert que la première fois. Pour les prochains push, vous pourrez simplement taper :

```
git push
```

Retournez sur la page de votre repository GitHub et actualisez : vous devez voir votre fichier `hello.js` apparaître en ligne !

11 git pull : récupérer les changements

Si un changement a été fait directement sur GitHub (par vous depuis un autre ordinateur, ou par quelqu'un d'autre), vous devez le récupérer sur votre ordinateur avant de continuer à travailler.

```
git pull
```

Règle d'or : faites toujours un `git pull` avant de commencer à travailler, pour être sûr d'avoir la dernière version du projet.

Résumé — Les commandes à retenir

Commande	À quoi ça sert
<code>git --version</code>	Vérifier que Git est installé
<code>git config --global user.name/ema</code>	Configurer votre identité (une seule fois)
<code>git init</code>	Transformer un dossier en dépôt Git
<code>git remote add origin <lien></code>	Lier le dossier local à un repository GitHub
<code>git status</code>	Voir l'état des fichiers (modifiés, prêts...)
<code>git add <fichier> / git add .</code>	Préparer un fichier (ou tous) avant le commit
<code>git commit -m "message"</code>	Enregistrer un instantané du projet
<code>git push</code>	Envoyer les commits vers GitHub
<code>git pull</code>	Récupérer les derniers changements depuis GitHub

Félicitations ! Vous savez maintenant créer un compte GitHub, créer un repository, le lier à votre ordinateur, et faire votre premier cycle complet : `add` → `commit` → `push` → `pull`. C'est exactement ce cycle que vous répéterez à chaque fois que vous travaillerez sur un projet.